<알파카앞발털파카>
<미스터 대박 디너 서비스>
설계 산출물

문서버전	2.0
문서 ID	SE-2025-002
최종변경일	2025-10-26
문서상태	릴리즈

요 약

미스터 대박 디너 서비스의 설계 산출물을 기술.
서브 시스템의 구성과 각 서브 시스템의 구조를 기술.
주요 산출물

- ✓ 아키텍처도
- ✓ 클래스도
- ✓ 교류도
- ✓ 상태도

표 1 문서 변경 기록

문서이름	<SE-알파카앞발털파카><미스터 대박 디너 서비스> 설계 산출물		
문서 ID	SE-2025-002		
버전		변경일	설명
1	1	2025-10-18	5개의 서브시스템에 대한 아키텍처도를 작성하였다.
	2	2025-10-19	클래스도, 교류도, 상태도를 작성하였다.
	3	2025-10-26	유즈케이스 변경에 따라 순차도를 수정하였다.
	4		
	5		
	6		
	7		
	8		
	9		
	10		
2	0	2025-10-26	릴리즈

1 개 요

1.1 목 적

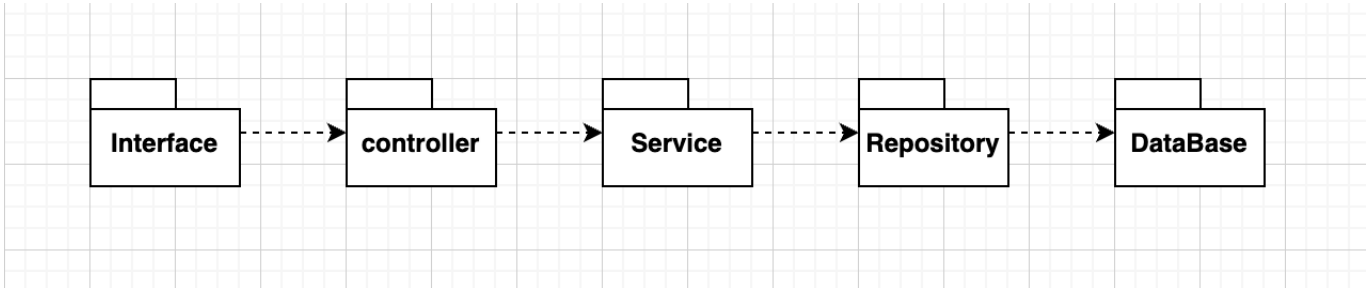
본 문서는 미스터 대박 서비스의 설계 산출물을 기술한다.

- ✓ 전체 시스템이 어떤 서브 시스템으로 구성되어 있는지 “아키텍처도”를 통해 보여준다.
- ✓ 서브 시스템이 어떤 클래스로 구성되어 있고 클래스들간에 어떤 관계가 있는지를 “클래스도”를 통해서 보여준다.
- ✓ 객체간에 발생하는 동적인 행위를 “교류도”를 통해서 보여준다.
- ✓ 한 객체의 상태 변화를 “상태도”를 통해서 보여준다.

1.2 참고 문헌

[1] 최은만, 소프트웨어 공학의 모든 것 (개정판), 생능출판사, 2024.

2 아키텍처도



전체 시스템은 아래의 서브 시스템으로 구성된다.

- Interface 시스템
- Controller 시스템
- Service 시스템
- Repository 시스템
- DataBase 시스템

Interface 시스템

- 고객과 직원이 상호작용하는 웹 기반 사용자 인터페이스를 담당한다. 고객은 웹 화면을 통해 주문, 장바구니, 재주문 등의 기능을 이용하며, 직원은 대시보드를 통해 주문 관리와 재고 현황을 확인한다.
- Next.js(React)와 TypeScript, Tailwind CSS를 사용하여 구현한다.

Controller 시스템

- 전체 요청 흐름의 진입점으로, Interface 또는 Voice 시스템으로부터 전달된 요청을 받아 Service 계층으로 전달한다.
- 클라이언트 요청을 검증하고, 비즈니스 로직에 따라 적절한 Service 메서드를 호출하며, 처리결과를 JSON 형태로 반환한다.

Service 시스템

- 비즈니스 로직의 핵심 부분을 담당하는 계층이다
- Controller로부터 전달받은 요청 데이터를 검증하고, 로직을 수행한 후 결과를 Repository계층에 전달하거나 반환한다.

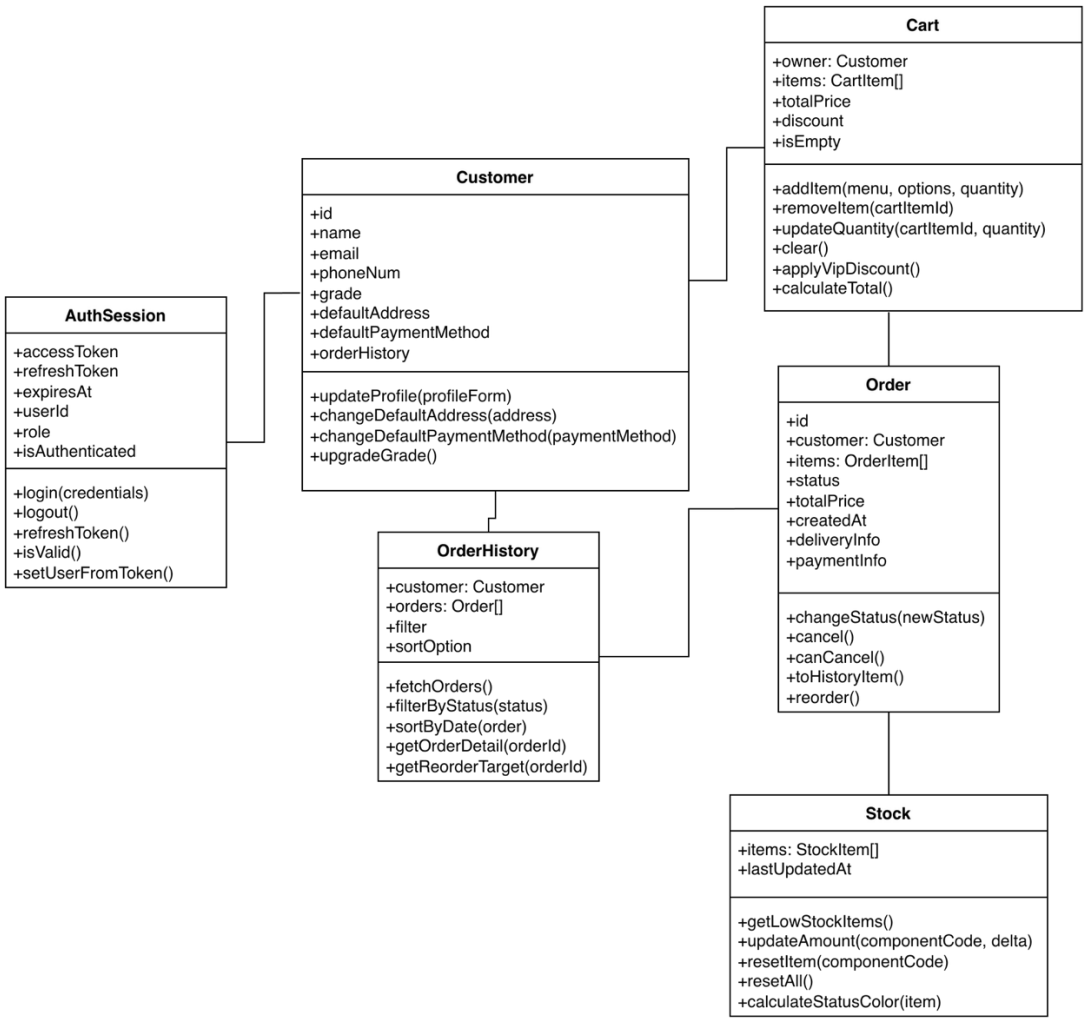
Repository 시스템

- 데이터베이스 접근을 담당하는 계층이다.
- Service 계층으로부터 전달받은 데이터를 저장, 수정, 조회, 삭제하는 기능을 수행한다.

DataBase 시스템

- 실제 데이터를 저장하는 계층으로, 데이터 무결성과 트랜잭션 안정성을 보장한다.
- MySQL기반의 관계형 데이터베이스를 사용한다.

3 세부 설계



3.1 “UI(interface)” 서브시스템

사용자 인터페이스(UI)에서 사용자의 입력과 상태를 관리하고, 서버와 통신하기 위한 데이터 모델을 정의합니다.

✓ AuthSession

■ 현재 로그인한 사용자의 인증 토큰과 세션 만료 시간을 관리하며, 로그인/로그아웃 기능을 수행합니다.

✓ Customer

■ 로그인한 사용자의 프로필 정보(이름, 연락처, 주소, 등급 등)를 관리하고 수정 요청을 처리합니다.

✓ Cart

■ 사용자가 선택한 메뉴 항목들을 임시 저장하며, 총 금액 계산 및 VIP 할인 적용, 장바구니 비우기 기능을 담당합니다.

✓ Order

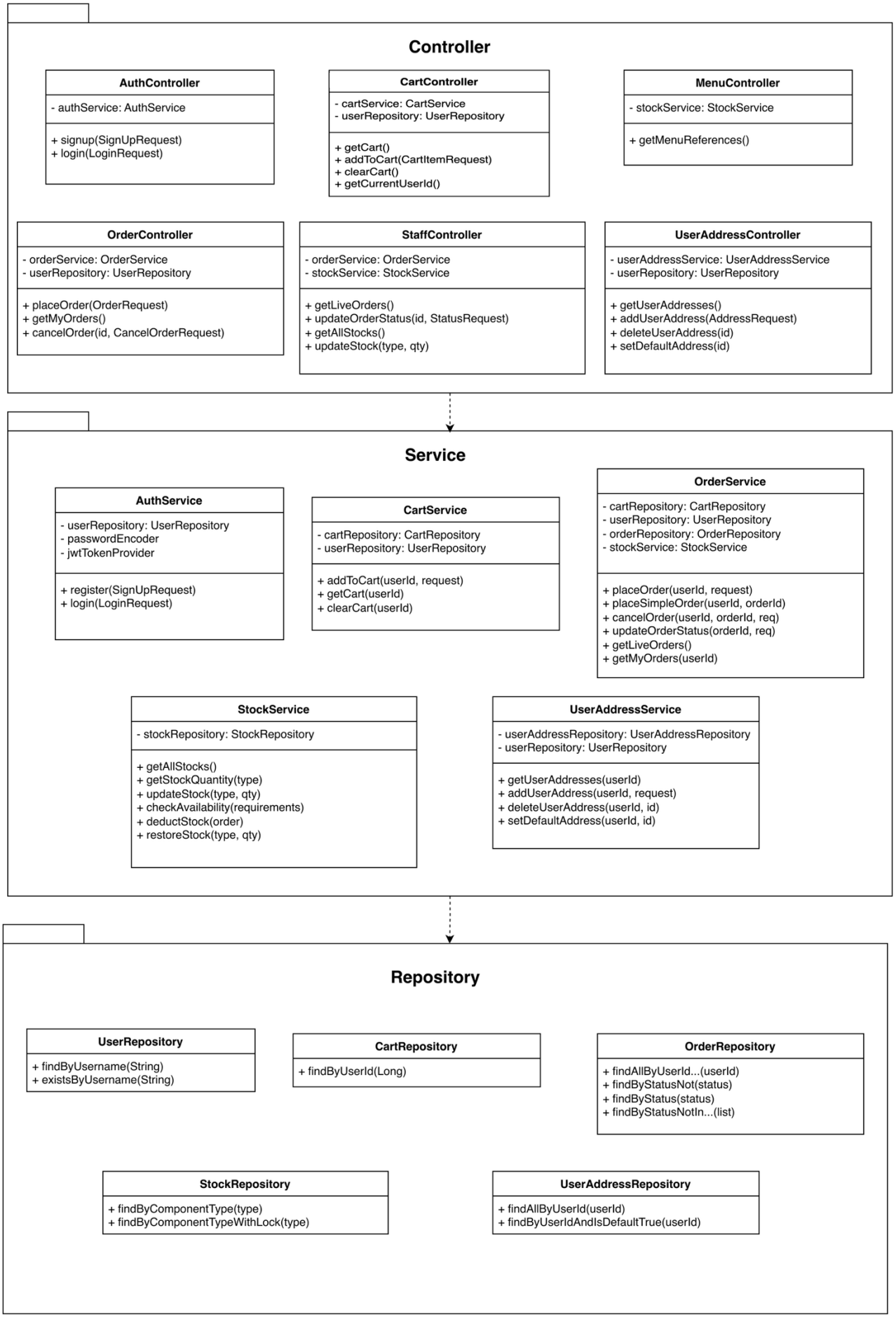
■ 생성된 주문의 상세 정보(상태, 결제 정보, 배송 정보 등)를 담고 있으며, 재주문이나 상태 변경 요청을 처리합니다.

✓ OrderHistory

■ 사용자의 과거 주문 내역 목록을 관리하며, 날짜별 정렬이나 상태별 필터링 기능을 제공합니다.

✓ Stock

■ (관리자용) 실시간 재고 현황을 모니터링하고, 부족 한 재고를 파악하여 시각적으로 표시합 니다.


Controller
AuthController
- authService: AuthService
+ signup(SignUpRequest)
+ login(LoginRequest)
CartController
- cartService: CartService
- userRepository: UserRepository
+ getCart()
+ addToCart(CartItemRequest)
+ clearCart()
+ getCurrentUserId()
MenuController
- stockService: StockService
+ getMenuReferences()
OrderController
- orderService: OrderService
- userRepository: UserRepository
+ placeOrder(OrderRequest)
+ getMyOrders()
+ cancelOrder(id, CancelOrderRequest)
StaffController
- orderService: OrderService
- stockService: StockService
+ getLiveOrders()
+ updateOrderStatus(id, StatusRequest)
+ getAllStocks()
+ updateStock(type, qty)
UserAddressController
- userAddressService: UserAddressService
- userRepository: UserRepository
+ getUserAddresses()
+ addUserAddress(AddressRequest)
+ deleteUserAddress(id)
+ setDefaultAddress(id)
Service
AuthService
- userRepository: UserRepository
- passwordEncoder
- jwtTokenProvider
+ register(SignUpRequest)
+ login(LoginRequest)
CartService
- cartRepository: CartRepository
- userRepository: UserRepository
+ addToCart(userId, request)
+ getCart(userId)
+ clearCart(userId)
OrderService
- cartRepository: CartRepository
- userRepository: UserRepository
- orderRepository: OrderRepository
- stockService: StockService
+ placeOrder(userId, request)
+ placeSimpleOrder(userId, orderId)
+ cancelOrder(userId, orderId, req)
+ updateOrderStatus(orderId, req)
+ getLiveOrders()
+ getMyOrders(userId)
StockService
- stockRepository: StockRepository
+ getAllStocks()
+ getStockQuantity(type)
+ updateStock(type, qty)
+ checkAvailability(requirements)
+ deductStock(order)
+ restoreStock(type, qty)
UserAddressService
- userAddressRepository: UserAddressRepository
- userRepository: UserRepository
+ getUserAddresses(userId)
+ addUserAddress(userId, request)
+ deleteUserAddress(userId, id)
+ setDefaultAddress(userId, id)
Repository
UserRepository
+ findByUsername(String)
+ existsByUsername(String)
CartRepository
+ findByUserId(Long)
OrderRepository
+ findAllByUserId...(userId)
+ findByStatusNot(status)
+ findByStatus(status)
+ findByStatusNotIn...(list)
StockRepository
+ findByComponentType(type)
+ findByComponentTypeWithLock(type)
UserAddressRepository
+ findAllByUserId(userId)
+ findByUserIdAndIsDefaultTrue(userId)

3.2 “Controller” 서브시스템

Controller 시스템은 **클라이언트(UI)의 요청을 가장 먼저 받아 처리하는 진입점**이다.
입력값 검증, 인증/인가, 예외 처리 등을 수행하고, 핵심 로직은 Service 계층으로 위임한다.
즉, UI와 Service를 연결하는 **중간 제어 계층**이다.

✓ AuthController
■ 사용자 회원가입 및 로그인 요청을 처리하고 인증 토큰을 발급합니다.
✓ CartController
■ 사용자의 장바구니 조회, 메뉴 추가, 비우기 등 장바구니 관련 요청을 처리합니다.
✓ MenuController
■ 메뉴 주문 시 필요한 옵션(디너 타입, 서빙 스타일) 및 가격 참조 데이터를 제공합니다.
✓ OrderController
■ 일반 고객의 주문 생성, 내 주문 내역 조회, 주문 취소 요청을 처리합니다..
✓ StaffController
■ 직원을 위한 실시간 주문 목록 조회, 주문 상태 변경, 재고 관리 기능을 제공합니다.
✓ UserAddressController
■ 사용자의 배송지 목록 조회, 추가, 삭제 및 기본 배송지 설정 요청을 처리합니다.

3.3 “Service” 서브시스템

Service 시스템은 **비즈니스 로직의 핵심을 담당하는 계층**이다.
Controller로부터 받은 요청을 실제 트랜잭션 단위로 처리하며,
도메인 규칙과 상태 전이 규칙(State Transition)을 관리한다.

✓ AuthService
■ 구체적인 비즈니스 로직을 수행하며, 트랜잭션 관리와 도메인 객체의 상태 변경을 담당합니다.
✓ CartService
■ 회원가입 시 중복 검사 및 정보 저장, 로그인 시 인증 및 JWT 토큰 생성을 수행합니다.
✓ OrderService
■ 주문 생성(재고 검증·결제), 상태 변경, 취소, 환불 등 주문 생명주기 전반의 핵심 로직을 처리합니다.
✓ StockService
■ 식자재 재고를 조회·수정하고, 주문 시 가용성 검증 및 재고 차감/복구 로직을 수행합니다.
✓ UserAddressService
■ 사용자의 배송지 정보를 등록, 조회, 삭제하고 기본 배송지를 관리합니다.

3.4 “Repository” 서브시스템

Repository 시스템은 **데이터베이스와 직접 연결되어 데이터를 관리하는 계층**이다.
Service 계층의 요청에 따라 데이터를 조회, 저장, 수정, 삭제하며
Spring Data JPA 기반으로 동작한다.

✓ **UserRepository**

■ 사용자 계정 정보(ID, 비밀번호, 권한 등)를 DB에서 조회하거나 저장합니다.

✓ **CartRepository**

■ 사용자별 장바구니 데이터를 DB에서 조회하거나 저장합니다.

✓ **OrderRepository**

■ 주문 내역을 저장하고, 상태별/사용자별 주문 목록을 조회합니다.

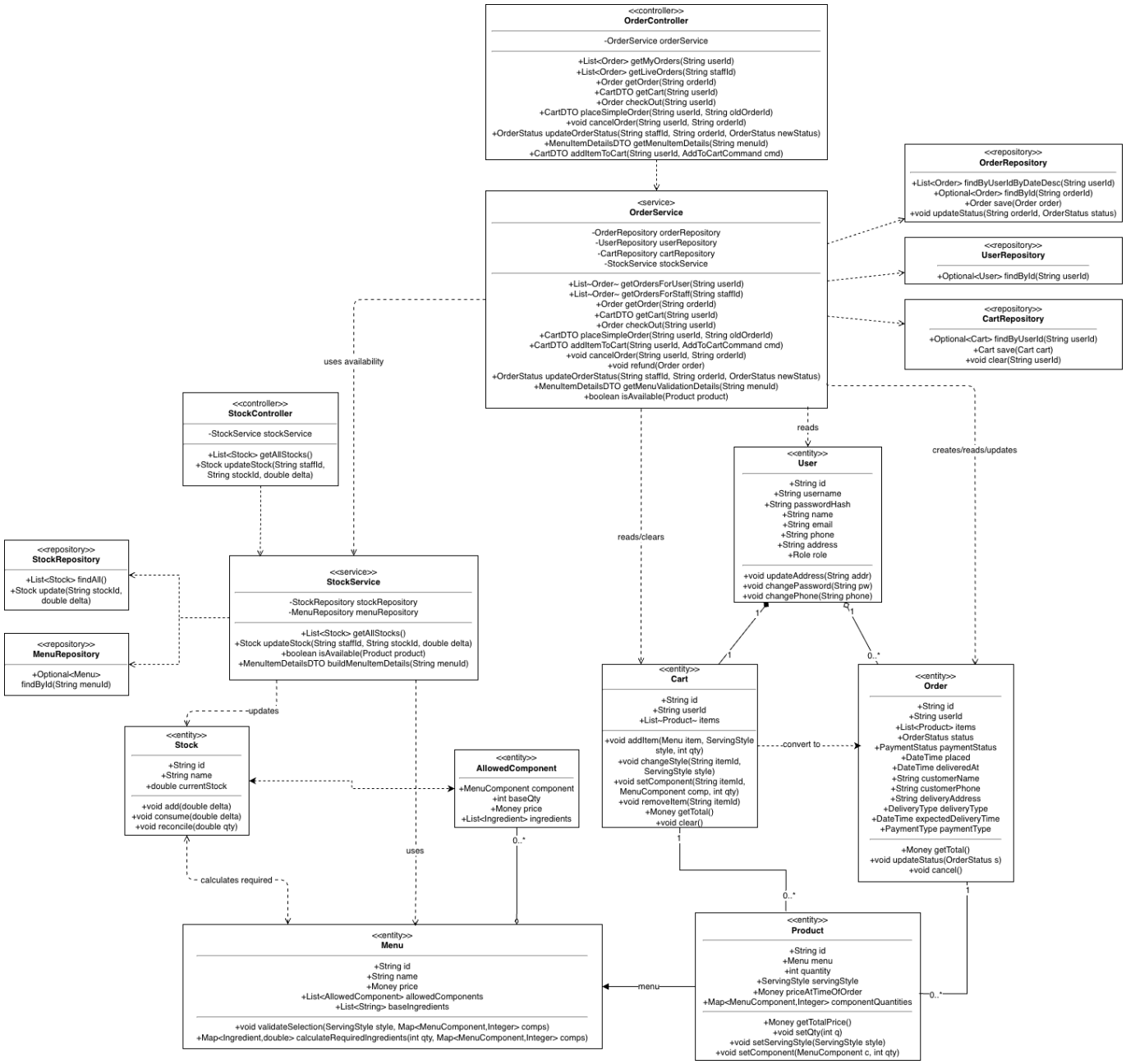
✓ **StockRepository**

■ 식자재 재고 정보를 조회하며, 동시성 제어를 위한 락(Lock) 기능을 포함합니다.

✓ **UserAddressRepository**

■ 사용자의 배송지 목록 및 기본 배송지 정보를 DB에서 관리합니다.

3.7 메인 클래스도



OrderController

- 고객과 직원의 주문 관련 요청을 처리한다.
- 사용자의 주문 목록, 주문 상세, 장바구니, 간편 주문, 주문 취소, 상태 변경 등을 담당한다.
- 메뉴 상세 조회와 장바구니 담기 기능을 제공한다.
- 내부적으로 OrderService를 호출해 비즈니스 로직을 수행한다.

StockController

- 직원이 사용하는 재고 관리 기능을 제공한다.
- 현재 재고 목록을 조회하거나 수량을 조정할 수 있다.
- StockService를 호출하여 재고의 증감 및 검증 로직을 수행한다.

OrderService

• 주문 관련 핵심 로직을 담당하는 서비스 계층이다.
• 주문 생성, 조회, 결제, 취소, 환불, 상태 변경 등 전 과정을 관리한다.
• 장바구니와 주문 내역을 연동하고, 메뉴와 재고의 가용성을 검증한다.
• OrderRepository, UserRepository, CartRepository, StockService 등을 협력자로 가진다.

StockService
• 재고 데이터의 조회와 수정, 메뉴 재료 가용성 검증을 담당한다.
• 주문 시 재고가 충분한지 확인하고, 결제 후 재고를 차감한다.
• 메뉴별 구성 요소를 바탕으로 필요한 재료량을 계산한다.
• StockRepository와 MenuRepository를 통해 실제 데이터를 조회하고 갱신한다.

User
• 시스템 내 고객 또는 직원의 계정 정보를 나타낸다.
• 이름, 아이디, 비밀번호, 이메일, 전화번호, 주소, 권한 등의 정보를 포함한다.
• 비밀번호, 전화번호, 주소 수정 기능을 제공한다.

Cart
• 고객이 주문 전 선택한 메뉴들을 임시로 저장하는 장바구니 역할을 한다.
• 메뉴 추가, 옵션 변경, 구성요소 수량 조정, 항목 삭제가 가능하다.
• 총 금액 계산 및 장바구니 초기화 기능을 포함한다.
• 주문 확정 시 Order 엔티티로 변환된다.

Product
• 주문 또는 장바구니 내 하나의 메뉴 항목을 표현한다.
• 메뉴, 수량, 스타일, 구성요소, 가격 등의 정보를 포함한다.
• 수량 및 스타일 변경, 컴포넌트 수량 조정 등의 기능을 제공한다.
• 메뉴 가격은 주문 시점의 스냅샷으로 저장된다.

Order
• 고객의 결제가 완료된 주문 정보를 관리한다.
• 주문 항목, 상태, 결제 상태, 고객 정보, 배달 방식 등을 포함한다.
• 주문 상태 변경 및 취소 기능을 제공한다.
• 주문 금액 계산 기능을 가진다.

Menu
• 주문 가능한 메뉴 정보를 정의한다.
• 기본 가격, 허용 가능한 옵션, 기본 재료를 포함한다.
• 선택된 옵션이 유효한지 검증하고 필요한 재료량을 계산한다.
• AllowedComponent 및 Stock과 연관된다.

AllowedComponent
- 메뉴에 추가할 수 있는 구성요소의 정보와 가격을 정의한다.
- 기본 수량과 재료 목록을 포함한다.
- 메뉴 선택 시 옵션 제약 검증에 활용된다.

Stock
- 식자재나 상품 재고를 나타낸다.
- 현재 수량을 보유하며, 재고 증가, 소모, 조정 기능을 제공한다.
- 주문 시 필요한 재료만큼 재고를 차감한다.
- StockService를 통해 주기적으로 관리된다.

OrderRepository
- 주문 데이터를 데이터베이스에 저장하고 조회한다.
- 사용자별 주문 목록 조회, 단건 조회, 상태 변경 등의 기능을 제공한다.
- OrderService에 의해 호출되어 주문 내역을 관리한다.

UserRepository
- 사용자 정보를 조회하고 인증 과정에서 활용된다.
- 아이디 기반 단건 조회 기능을 제공한다.
- 회원가입 및 로그인 시 사용자 정보 접근에 사용된다.

CartRepository
- 사용자별 장바구니 데이터를 저장하고 불러온다.
- 장바구니 조회, 저장, 초기화 기능을 제공한다.
- 주문 완료 시 장바구니를 비운다.

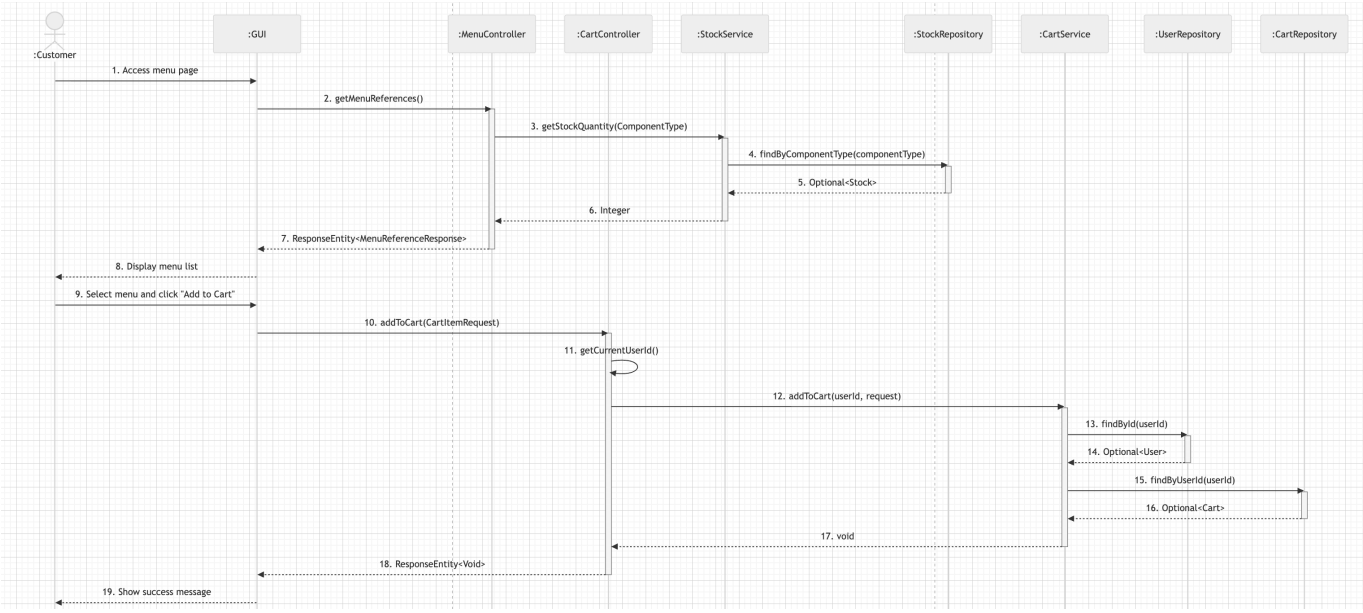
StockRepository
- 재고 데이터를 데이터베이스에서 조회하고 수정한다.
- 전체 재고 목록 확인 및 수량 변경 기능을 제공한다.
- StockService를 통해 호출되어 실시간 재고 상태를 유지한다.

MenuRepository
- 메뉴 정의 데이터를 조회한다.
- 메뉴 아이디로 검색해 상세 정보를 가져온다.
- 메뉴 선택 시 검증 및 상세 표시 기능에 사용된다.

Role
- 사용자 권한을 구분하기 위한 열거형이다.
- CUSTOMER, KITCHEN_STAFF, DELIVERY_STAFF로 구성된다.
- 로그인 및 접근 제어 로직에서 사용된다.

4 교류도

4.1 New Order Process

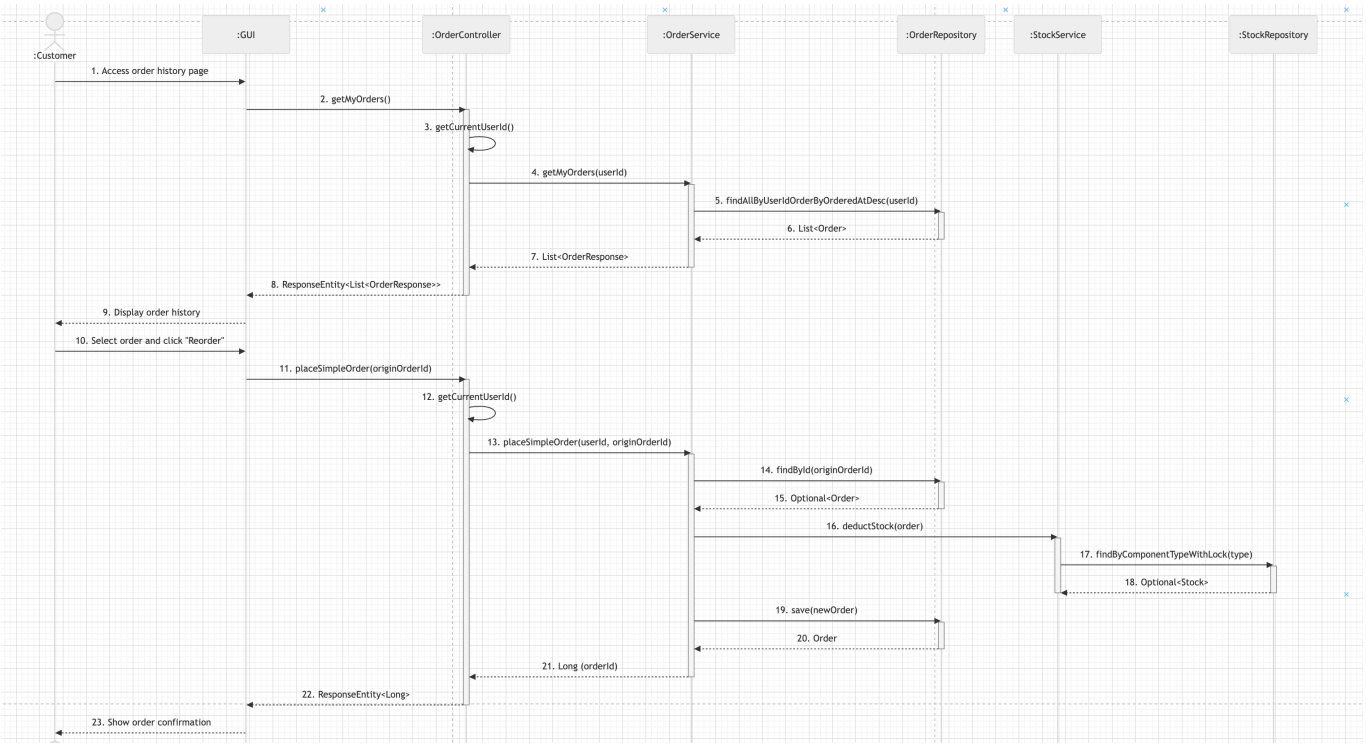


MenuController 클래스는 StockService의 여러 메서드를 호출하여 메뉴 참조 데이터 조회 기능을 수행하고, CartController 클래스는 CartService의 여러 메서드를 호출하여 장바구니 추가 기능을 수행한다. Customer가 요청하는 메뉴 조회, 장바구니 담기 등의 명령은 GUI를 통해 Controller로 전달되며, Controller는 이를 서비스 계층으로 넘겨 시스템 내부에서 처리한다.

1. Access menu page는 고객이 메뉴 페이지에 접속하여 메뉴 목록을 요청한다.
2. getMenuReferences()는 GUI가 MenuController에 메뉴 참조 데이터 조회를 요청한다.
3. getStockQuantity()는 MenuController가 StockService에 각 구성품의 재고 수량 조회를 요청한다.
4. findByComponentType(componentType)는 StockService가 StockRepository를 통해 재고 정보를 조회한다.
5. Optional<Stock>는 StockRepository가 조회 결과를 StockService에 반환한다.
6. Integer는 StockService가 재고 수량을 MenuController에 반환한다.
7. ResponseEntity<MenuReferenceResponse>는 MenuController가 메뉴 참조 데이터와 재고 정보를 GUI로 반환한다.
8. Display menu list는 GUI가 고객에게 메뉴 목록과 재고 정보를 표시한다.
9. Select menu and click "Add to Cart"는 고객이 메뉴를 선택하고 수량을 입력한 후 장바구에 추가한다。
10. addToCart(CartItemRequest)는 GUI가 CartController에 장바구니 추가 요청을 보낸다.
11. getCurrentUserId()는 CartController가 현재 사용자 ID를 조회한다.
12. addToCart(userId, request)는 CartController가 CartService에 장바구니 추가를 위임한다.
13. findById(userId)는 CartService가 UserRepository를 통해 사용자 정보를 조회한다.
14. Optional<User>는 UserRepository가 조회된 사용자를 반환한다.
15. findByUserId(userId)는 CartService가 CartRepository에 사용자 장바구니 조회를 요청한다.
16. Optional<Cart>는 CartRepository가 조회된 장바구니를 반환한다.
17. void는 CartService가 처리 완료를 CartController에 반환한다.
18. ResponseEntity<Void>는 CartController가 성공 응답을 GUI로 전달한다.
19. Show success message는 GUI가 고객에게 추가 완료를 알린다.

즉, Customer의 입력은 GUI → MenuController/CartController → StockService/CartService → Repository의 흐름으로 전달되며, MenuController는 메뉴 조회 기능을, CartController는 장바구니 추가 기능의 중심 역할을 수행한다.

4.2 Simple Order Process



OrderController 클래스는 OrderService의 여러 메서드를 호출하여 과거 주문 조회 및 빠른 재주문 기능을 수행한다. Customer가 요청하는 주문 내역 조회, 재주문 등의 명령은 GUI를 통해 Controller로 전달되며, Controller는 이를 서비스 계층으로 넘겨 시스템 내부에서 처리한다.

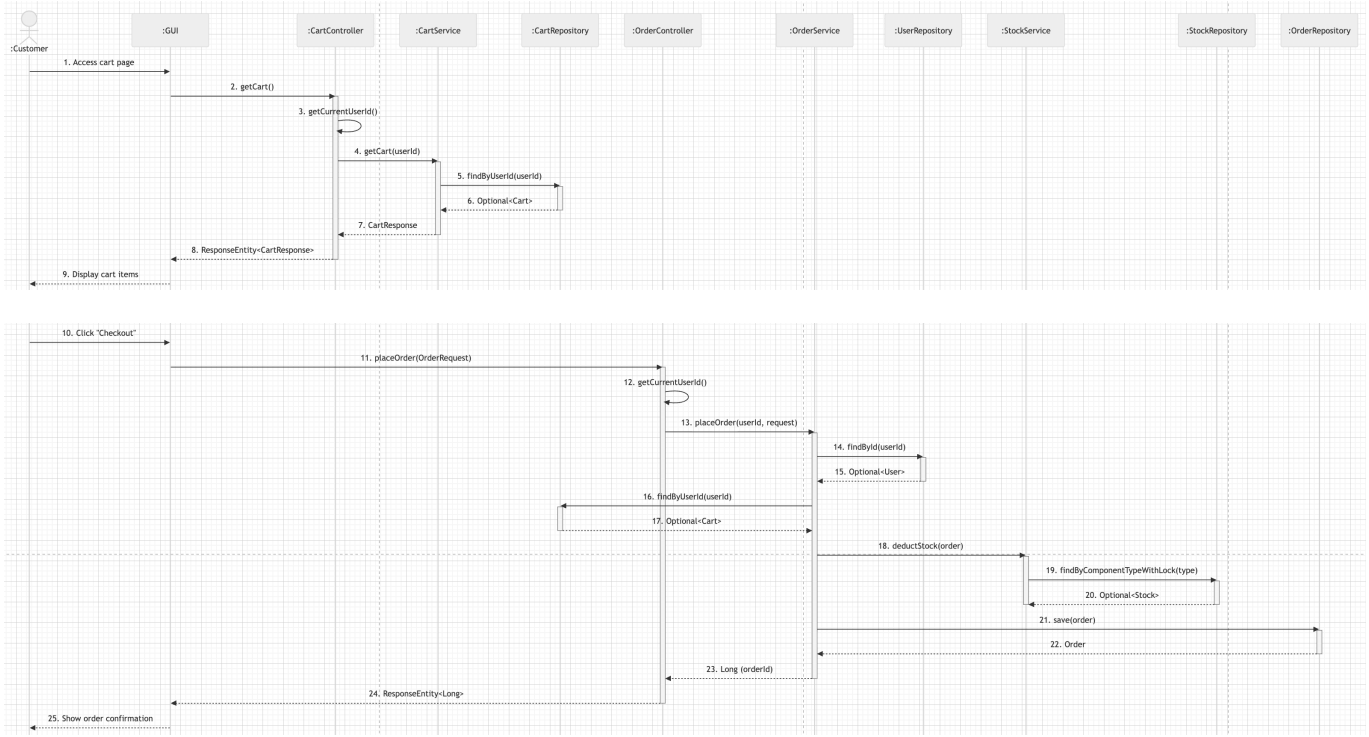
1. Access order history page는 고객이 주문 내역 페이지에 접속하여 과거 주문 목록을 요청한다.
2. getMyOrders()는 GUI가 OrderController에 주문 내역 조회를 요청한다.
3. getCurrentUserId()는 OrderController가 현재 사용자 ID를 조회한다.
4. getMyOrders(userId)는 OrderController가 OrderService에 주문 내역 조회를 위임한다.
5. findAllByUserIdOrderByOrderedAtDesc(userId)는 OrderService가 OrderRepository를 통해 사용자의 모든 주문을 조회한다.
6. List<Order>는 OrderRepository가 조회된 주문 목록을 반환한다.
7. List<OrderResponse>는 OrderService가 주문 목록을 OrderController에 반환한다.
8. ResponseEntity<List<OrderResponse>>는 OrderController가 주문 내역을 GUI로 전달한다.
9. Display order history는 GUI가 고객에게 과거 주문 내역을 표시한다.
10. Select order and click "Reorder"는 고객이 재주문할 주문을 선택하고 재주문 버튼을 클릭한다.
11. placeSimpleOrder(originOrderId)는 GUI가 OrderController에 빠른 재주문 요청을 보낸다.
12. getCurrentUserId()는 OrderController가 현재 사용자 ID를 조회한다.
13. placeSimpleOrder(userId, originOrderId)는 OrderController가 OrderService에 재주문을 위임한다.
14. findById(originOrderId)는 OrderService가 OrderRepository를 통해 원본 주문을 조회한다.
15. Optional<Order>는 OrderRepository가 조회된 원본 주문을 반환한다.
16. deductStock(order)는 OrderService가 StockService에 재고 차감을 요청한다.
17. findByComponentTypeWithLock(type)는 StockService가 StockRepository를 통해 재고를 조회하고 차감한다.
18. Optional<Stock>는 StockRepository가 조회된 재고를 반환한다.
19. save(newOrder)는 OrderService가 OrderRepository를 통해 새 주문을 저장한다.
20. Order는 OrderRepository가 저장된 주문을 반환한다.
21. Long (orderId)는 OrderService가 생성된 주문 ID를 OrderController에 반환한다.

22. ResponseEntity<Long>는 OrderController가 주문 ID를 GUI로 전달한다.

23. Show order confirmation는 GUI가 고객에게 주문 완료를 알린다.

즉, Customer의 입력은 GUI → OrderController → OrderService → OrderRepository/StockService의 흐름으로 전달되며, OrderController는 주문 내역 조회와 빠른 재주문 기능의 중심 역할을 수행한다.

4.3 Cart



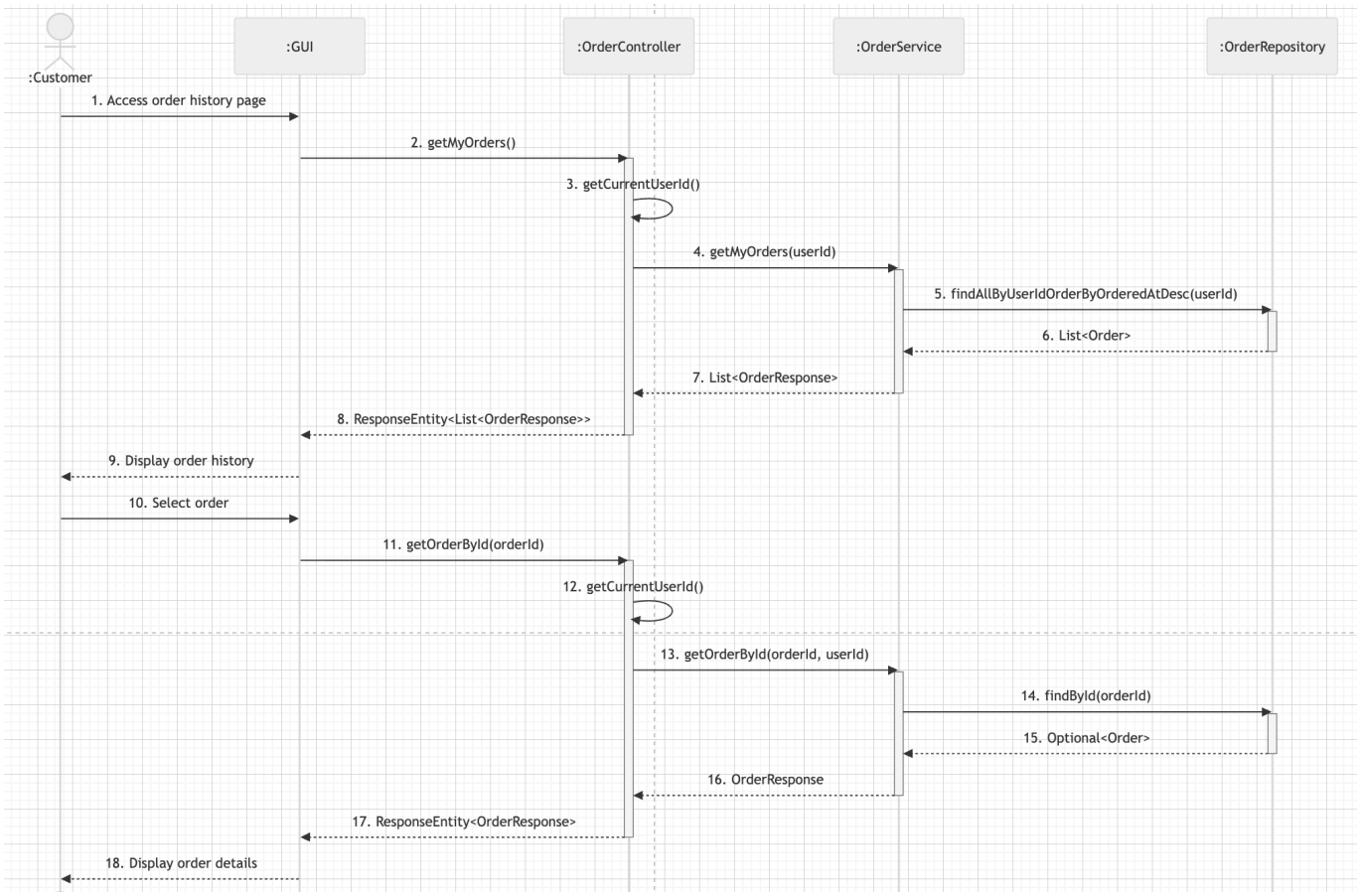
CartController 클래스는 CartService의 여러 메서드를 호출하여 장바구니 조회 기능을 수행하고, OrderController 클래스는 OrderService를 호출하여 결제 기능을 수행한다. Customer가 요청하는 장바구니 조회, 결제 등의 명령은 GUI를 통해 Controller로 전달되며, Controller는 이를 서비스 계층으로 넘겨 시스템 내부에서 처리한다.

1. Access cart page는 고객이 장바구니 페이지에 접속하여 장바구니 내용을 요청한다.
2. getCart()는 GUI가 CartController에 장바구니 조회를 요청한다.
3. getCurrentUserId()는 CartController가 현재 사용자 ID를 조회한다.
4. getCart(userId)는 CartController가 CartService에 장바구니 조회를 위임한다.
5. findByUserId(userId)는 CartService가 CartRepository에 사용자 장바구니 조회를 요청한다.
6. Optional<Cart>는 CartRepository가 조회된 장바구니를 반환한다.
7. CartResponse는 CartService가 장바구니 정보를 CartController에 반환한다.
8. ResponseEntity<CartResponse>는 CartController가 장바구니 내용을 GUI로 전달한다.
9. Display cart items는 GUI가 고객에게 장바구니 내역을 표시한다.
10. Click "Checkout"는 고객이 결제 버튼을 클릭한다.
11. placeOrder(OrderRequest)는 GUI가 OrderController에 주문 생성 요청을 보낸다.
12. getCurrentUserId()는 OrderController가 현재 사용자 ID를 조회한다.
13. placeOrder(userId, request)는 OrderController가 OrderService에 주문 생성을 위임한다.
14. findById(userId)는 OrderService가 UserRepository를 통해 사용자를 조회한다.
15. Optional<User>는 UserRepository가 조회된 사용자를 반환한다.
16. findByUserId(userId)는 OrderService가 CartRepository를 통해 장바구니를 조회한다.
17. Optional<Cart>는 CartRepository가 조회된 장바구니를 반환한다.
18. deductStock(order)는 OrderService가 StockService에 재고 차감을 요청한다.
19. findByComponentTypeWithLock(type)는 StockService가 StockRepository를 통해 재고를 조회하고 차감한다.
20. Optional<Stock>는 StockRepository가 조회된 재고를 반환한다.
21. save(order)는 OrderService가 OrderRepository를 통해 주문을 저장한다.
22. Order는 OrderRepository가 저장된 주문을 반환한다.

23. Long (orderId)는 OrderService가 생성된 주문 ID를 OrderController에 반환한다.
24. ResponseEntity<Long>는 OrderController가 주문 ID를 GUI로 전달한다.
25. Show order confirmation는 GUI가 고객에게 주문 완료를 알린다.

즉, Customer의 입력은 GUI → CartController/OrderController → CartService/OrderService → Repository의 흐름으로 전달되며, CartController는 장바구니 조회 기능을, OrderController는 주문 생성 기능의 중심 역할을 수행한다.

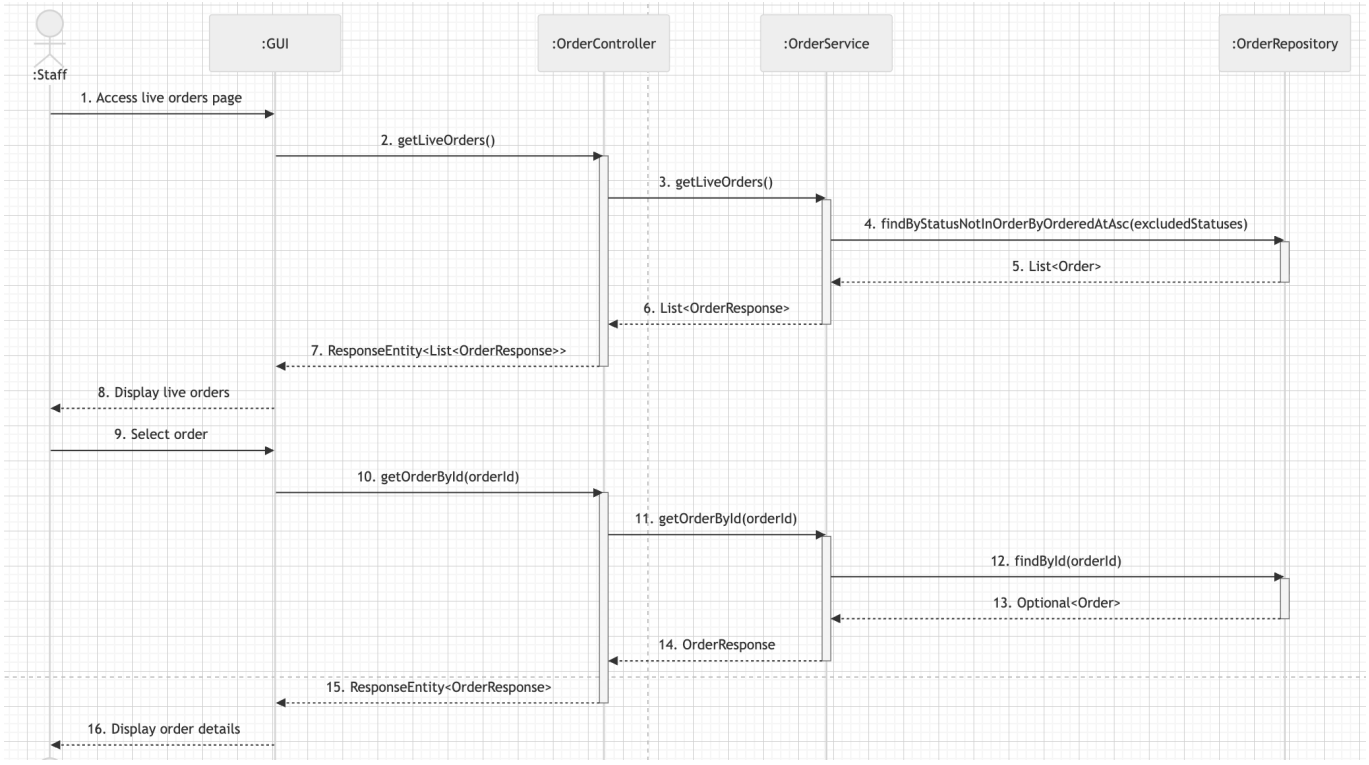
4.4 View Orders



OrderController 클래스는 OrderService의 여러 메서드를 호출하여 고객의 주문 내역 조회 기능을 수행한다. Customer가 요청하는 주문 내역 조회 명령은 GUI를 통해 Controller로 전달되며, Controller는 이를 서비스 계층으로 넘겨 시스템 내부에서 처리한다.

1. Access order history page는 고객이 주문 내역 페이지에 접속하여 자신의 주문 목록을 요청한다.
2. getMyOrders()는 GUI가 OrderController에 주문 내역 조회를 요청한다.
3. getCurrentUserId()는 OrderController가 현재 사용자 ID를 조회한다.
4. getMyOrders(userId)는 OrderController가 OrderService에 주문 내역 조회를 위임한다.
5. findAllByUserIdOrderByOrderedAtDesc(userId)는 OrderService가 OrderRepository를 통해 사용자의 모든 주문을 조회한다.
6. List<Order>는 OrderRepository가 조회된 주문 목록을 반환한다.
7. List<OrderResponse>는 OrderService가 주문 목록을 OrderController에 반환한다.
8. ResponseEntity<List<OrderResponse>>는 OrderController가 주문 내역을 GUI로 전달한다.
9. Display order history는 GUI가 고객에게 주문 내역을 표시한다.
10. Select order는 고객이 특정 주문을 선택하여 상세 내역을 요청한다.
11. getOrderById(orderId)는 GUI가 OrderController에 주문 상세 조회를 요청한다.
12. getCurrentUserId()는 OrderController가 현재 사용자 ID를 조회한다.
13. getOrderById(orderId, userId)는 OrderController가 OrderService에 주문 상세 조회를 위임한다.
14. findById(orderId)는 OrderService가 OrderRepository를 통해 주문을 조회한다.
15. Optional<Order>는 OrderRepository가 조회된 주문을 반환한다.
16. OrderResponse는 OrderService가 주문 상세 정보를 OrderController에 반환한다.
17. ResponseEntity<OrderResponse>는 OrderController가 주문 상세 내역을 GUI로 전달한다.
18. Display order details는 GUI가 고객에게 주문 상세 정보를 표시한다.

즉, Customer의 입력은 GUI → OrderController → OrderService → OrderRepository의 흐름으로 전달되며, OrderController는 고객의 주문 내역 조회 및 상세 조회 기능의 중심 역할을 수행한다.

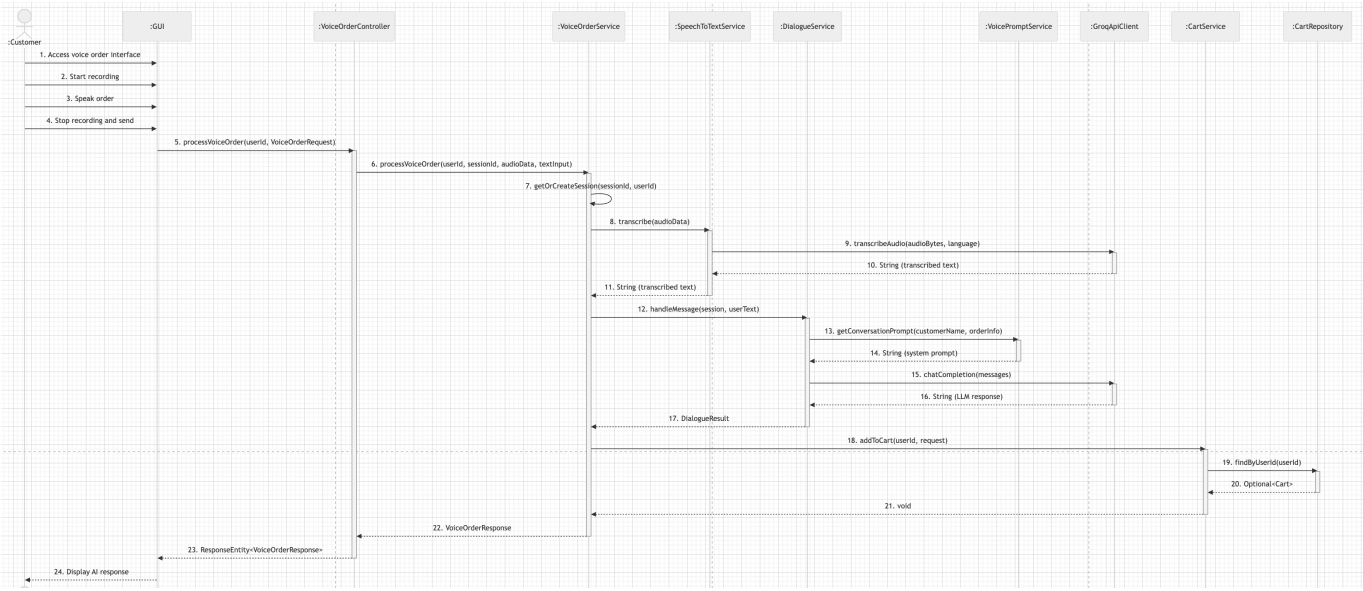


OrderController 클래스는 OrderService의 여러 메서드를 호출하여 실시간 진행 중인 주문 조회 기능을 수행한다. Staff가 요청하는 실시간 주문 조회 명령은 GUI를 통해 Controller로 전달되며, Controller는 이를 서비스 계층으로 넘겨 시스템 내부에서 처리한다.

1. Access live orders page는 직원이 실시간 주문 관리 페이지에 접속하여 진행중인 주문 목록을 요청한다.
2. getLiveOrders()는 GUI가 OrderController에 진행중인 주문 조회를 요청한다.
3. getLiveOrders()는 OrderController가 OrderService에 진행중인 주문 조회를 위임한다.
4. findByStatusNotInOrderByOrderedAtAsc(excludedStatuses)는 OrderService가 OrderRepository를 통해 완료되지 않은 주문을 조회한다.
5. List<Order>는 OrderRepository가 조회된 진행중인 주문 목록을 반환한다.
6. List<OrderResponse>는 OrderService가 주문 목록을 OrderController에 반환한다.
7. ResponseEntity<List<OrderResponse>>는 OrderController가 진행중인 주문 내역을 GUI로 전달한다.
8. Display live orders는 GUI가 직원에게 진행중인 주문 목록을 표시한다.
9. Select order는 직원이 특정 주문을 선택하여 상세 내역을 요청한다.
10. getOrderById(orderId)는 GUI가 OrderController에 주문 상세 조회를 요청한다.
11. getOrderById(orderId)는 OrderController가 OrderService에 주문 상세 조회를 위임한다.
12. findById(orderId)는 OrderService가 OrderRepository를 통해 주문을 조회한다.
13. Optional<Order>는 OrderRepository가 조회된 주문을 반환한다.
14. OrderResponse는 OrderService가 주문 상세 정보를 OrderController에 반환한다.
15. ResponseEntity<OrderResponse>는 OrderController가 주문 상세 내역을 GUI로 전달한다.
16. Display order details는 GUI가 직원에게 주문 상세 정보를 표시한다.

즉, Staff의 입력은 GUI → OrderController → OrderService → OrderRepository의 흐름으로 전달되며, OrderController는 실시간 주문 조회 및 상세 조회 기능의 중심 역할을 수행한다.

4.5 Voice Order



VoiceOrderController 클래스는 VoiceOrderService의 여러 메서드를 호출하여 음성 주문 처리 기능을 수행한다. Customer가 요청하는 음성 입력은 GUI를 통해 Controller로 전달되며, Controller는 이를 서비스 계층으로 넘겨 시스템 내부에서 처리한다.

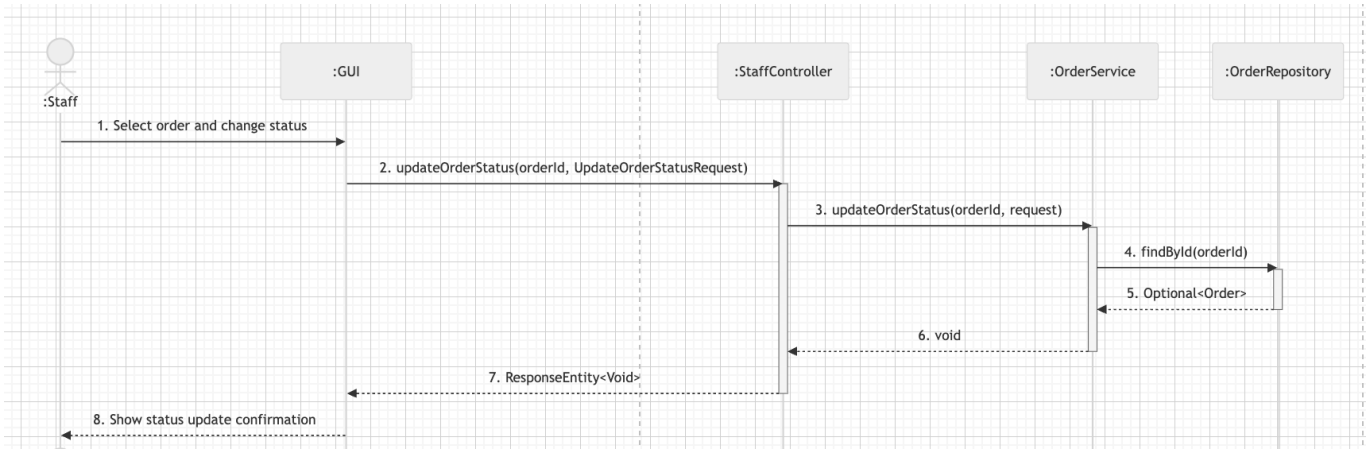
1. Access voice order interface는 고객이 음성 주문 인터페이스에 접속한다.
2. Start recording는 고객이 음성 녹음을 시작한다.
3. Speak order는 고객이 음성으로 주문 내용을 말한다.
4. Stop recording and send는 고객이 녹음을 종료하고 음성 데이터를 전송한다.
5. processVoiceOrder(userId, VoiceOrderRequest)는 GUI가 VoiceOrderController에 음성 주문 처리를 요청한다.
6. processVoiceOrder(userId, sessionId, audioData, textInput)는 VoiceOrderController가 VoiceOrderService에 음성 주문 처리를 위임한다.
7. getOrCreateSession(sessionId, userId)는 VoiceOrderService가 대화 세션을 조회하거나 생성한다.
8. transcribe(audioData)는 VoiceOrderService가 SpeechToTextService에 음성을 텍스트로 변환을 요청한다.
9. transcribeAudio(audioBytes, language)는 SpeechToTextService가 GroqApiClient에 Whisper API 호출을 요청한다.
10. String (transcribed text)는 GroqApiClient가 변환된 텍스트를 반환한다.
11. String (transcribed text)는 SpeechToTextService가 변환된 텍스트를 VoiceOrderService에 반환한다.
12. handleMessage(session, userText)는 VoiceOrderService가 DialogueService에 대화 처리를 요청한다.
13. getConversationPrompt(customerName, orderInfo)는 DialogueService가 VoicePromptService에 프롬프트 생성을 요청한다.
14. String (system prompt)는 VoicePromptService가 생성된 프롬프트를 반환한다.
15. chatCompletion(messages)는 DialogueService가 GroqApiClient에 LLM 호출을 요청한다.
16. String (LLM response)는 GroqApiClient가 LLM 응답을 반환한다.
17. DialogueResult는 DialogueService가 대화 처리 결과를 VoiceOrderService에 반환한다.
18. addToCart(userId, request)는 VoiceOrderService가 CartService에 장바구니 추가를 요청한다.
19. findByUserId(userId)는 CartService가 CartRepository를 통해 장바구니를 조회한다.
20. Optional<Cart>는 CartRepository가 조회된 장바구니를 반환한다.
21. void는 CartService가 처리 완료를 VoiceOrderService에 반환한다.
22. VoiceOrderResponse는 VoiceOrderService가 음성 주문 응답을 VoiceOrderController에 반환한다.

23. ResponseEntity<VoiceOrderResponse>는 VoiceOrderController가 응답을 GUI로 전달한다.

24. Display AI response는 GUI가 고객에게 AI 응답을 표시한다.

즉, Customer의 입력은 GUI → VoiceOrderController → VoiceOrderService → SpeechToTextService / DialogueService / CartService → GroqApiClient / CartRepository의 흐름으로 전달되며, VoiceOrderController는 음성 주문 처리 기능의 중심 역할을 수행한다.

4.6 Order Management

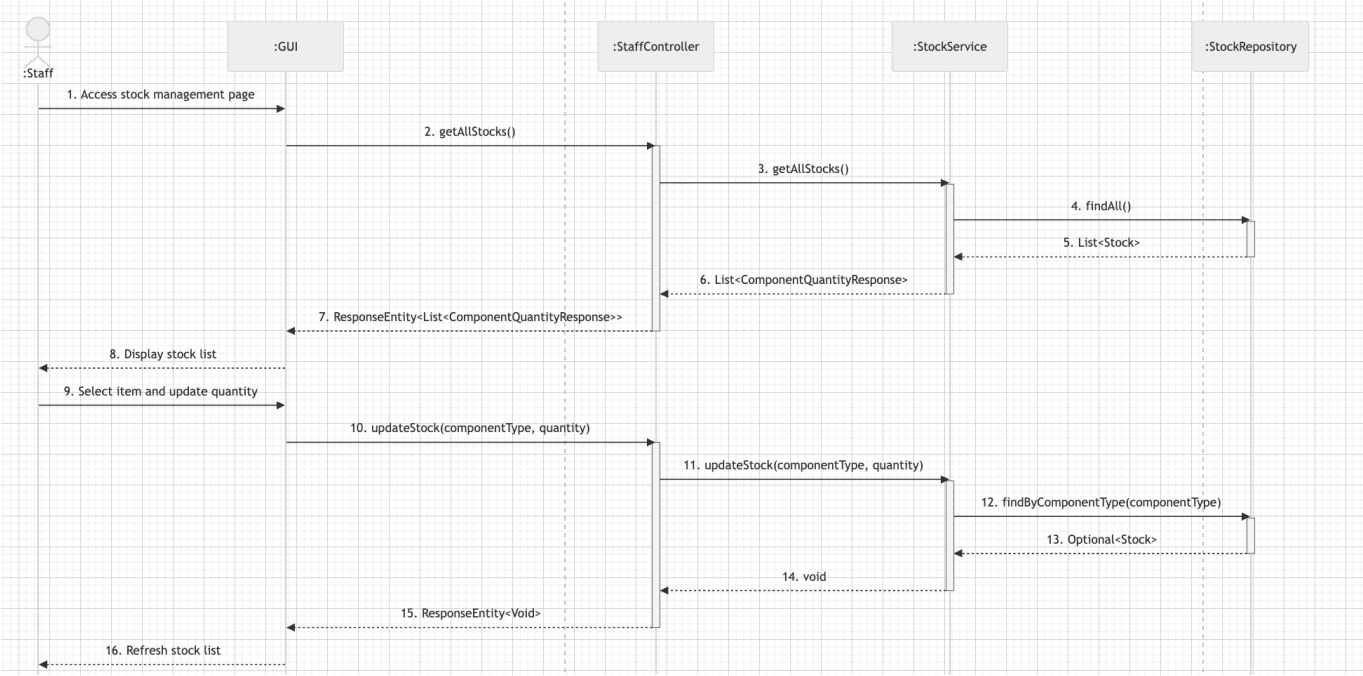


StaffController 클래스는 OrderService의 여러 메서드를 호출하여 주문 상태 변경 기능을 수행한다. Staff가 요청하는 주문 상태 변경 명령은 GUI를 통해 Controller로 전달되며, Controller는 이를 서비스 계층으로 넘겨 시스템 내부에서 처리한다.

1. Select order and change status는 직원이 특정 주문을 선택하고 상태를 변경한다.
2. updateOrderStatus(orderId, UpdateOrderStatusRequest)는 GUI가 StaffController에 주문 상태 변경을 요청한다.
3. updateOrderStatus(orderId, request)는 StaffController가 OrderService에 주문 상태 변경을 위임한다.
4. findById(orderId)는 OrderService가 OrderRepository를 통해 주문을 조회한다.
5. Optional<Order>는 OrderRepository가 조회된 주문을 반환한다.
6. void는 OrderService가 처리 완료를 StaffController에 반환한다.
7. ResponseEntity<Void>는 StaffController가 성공 응답을 GUI로 전달한다.
8. Show status update confirmation는 GUI가 직원에게 상태 변경 완료를 알린다.

즉, Staff의 입력은 GUI → StaffController → OrderService → OrderRepository의 흐름으로 전달되며, StaffController는 주문 상태 변경 기능의 중심 역할을 수행한다.

4.7 Stock Management

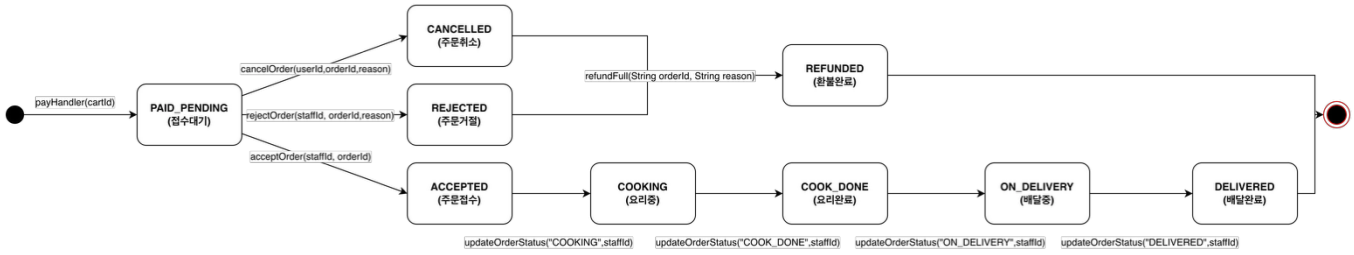


StaffController 클래스는 StockService의 여러 메서드를 호출하여 재고 조회 및 수정 기능을 수행한다. Staff가 요청하는 재고 관리 명령은 GUI를 통해 Controller로 전달되며, Controller는 이를 서비스 계층으로 넘겨 시스템 내부에서 처리한다.

1. Access stock management page는 직원이 재고 관리 페이지에 접속하여 재고 목록을 요청한다.
2. getAllStocks()는 GUI가 StaffController에 재고 조회를 요청한다.
3. getAllStocks()는 StaffController가 StockService에 재고 조회를 위임한다.
4. findAll()는 StockService가 StockRepository를 통해 모든 재고를 조회한다.
5. List<Stock>는 StockRepository가 조회된 재고 목록을 반환한다.
6. List<ComponentQuantityResponse>는 StockService가 재고 목록을 StaffController에 반환한다.
7. ResponseEntity<List<ComponentQuantityResponse>>는 StaffController가 재고 목록을 GUI로 전달한다.
8. Display stock list는 GUI가 직원에게 재고 목록을 표시한다.
9. Select item and update quantity는 직원이 특정 재고를 선택하고 수량을 변경한다.
10. updateStock(componentType, quantity)는 GUI가 StaffController에 재고 수정을 요청한다.
11. updateStock(componentType, quantity)는 StaffController가 StockService에 재고 수정을 위임한다.
12. findByComponentType(componentType)는 StockService가 StockRepository를 통해 재고를 조회한다.
13. Optional<Stock>는 StockRepository가 조회된 재고를 반환한다.
14. void는 StockService가 처리 완료를 StaffController에 반환한다.
15. ResponseEntity<Void>는 StaffController가 성공 응답을 GUI로 전달한다.
16. Refresh stock list는 GUI가 직원에게 갱신된 재고 목록을 표시한다.

즉, Staff의 입력은 GUI → StaffController → StockService → StockRepository의 흐름으로 전달되며, StaffController는 재고 관리 기능의 중심 역할을 수행한다.

5. Order 객체 상태도



Order는 주문의 상태를 나타내는 클래스이다. 주문은 다음 아홉 개의 상태 중 하나를 가진다.

✓ PAID_PENDING (접수대기)

- 고객이 결제를 완료하면 “UI” 시스템에서 payHandler(cartId)를 통해 “Server” 시스템으로 결제 데이터를 전송한다.
- 서버는 이를 기반으로 새로운 주문(Order)을 생성하고, 주문 상태를 **접수대기(PAID_PENDING)** 상태로 변경한다.
- 고객이 주문을 취소할 경우 “UI”에서 cancelOrder(userId, orderId, reason)을 통해 서버에 전송하며, 주문 상태는 **CANCELLED(주문취소)** 로 변경된다.

✓ ACCEPTED (주문접수)

- 직원이 주문을 수락하면 “UI” 시스템에서 acceptOrder(staffId, orderId)를 통해 “Server” 시스템으로 데이터를 전송하여 주문 상태를 **주문접수(ACCEPTED)** 로 변경한다.
- 만약 주문을 거절해야 할 경우, rejectOrder(staffId, orderId, reason)을 통해 주문 상태를 **REJECTED(주문거절)** 로 변경한다.

✓ COOKING (요리중)

- 주문이 접수된 후, 직원이 요리를 시작할 때 “UI” 시스템에서 updateOrderStatus("COOKING", staffId)를 통해 서버로 전달한다.
- 서버는 주문 상태를 **요리중(COOKING)** 으로 변경한다.

✓ COOK_DONE (요리완료)

- 직원이 요리를 완료하면 “UI” 시스템에서 updateOrderStatus("COOK_DONE", staffId)를 통해 “Server” 시스템으로 데이터를 전송하여 주문 상태를 **요리완료(COOK_DONE)** 로 변경한다.

✓ ON_DELIVERY (배달중)

- 배달원이 배달을 시작하면 “UI” 시스템에서 updateOrderStatus("ON_DELIVERY", staffId)를 통해 “Server” 시스템으로 데이터를 전송하여 주문 상태를 **배달중(ON_DELIVERY)** 으로 변경한다.

✓ DELIVERED (배달완료)

- 배달원이 고객에게 상품을 전달한 후, “UI” 시스템에서 updateOrderStatus("DELIVERED", staffId)를 통해 “Server” 시스템으로 데이터를 전송하여 주문 상태를 **배달완료(DELIVERED)** 로 변경한다.

✓ CANCELLED (주문취소)

- 고객이 결제 후 주문을 직접 취소하면 “UI” 시스템에서 cancelOrder(userId, orderId, reason)를 통해 서버로 전송한다.
- 서버는 주문 상태를 CANCELLED로 변경하고, 필요 시 refundFull(orderId, reason)을 호출하여 REFUNDED(환불완료) 상태로 이동시킨다.

✓ REJECTED (주문거절)

- 직원이 주문을 거절한 경우, rejectOrder(staffId, orderId, reason)을 통해 상태를 REJECTED로 변경한다.
- 이후 refundFull(orderId, reason)을 통해 고객에게 전액 환불 처리하여 상태를 REFUNDED(환불완료)로 전환한다.

✓ REFUNDED (환불완료)

- 주문이 취소(CANCELLED) 또는 거절(REJECTED)되어 환불이 완료된 상태이다.
- 서버에서는 refundFull(orderId, reason) 호출을 통해 환불 절차를 완료한다.